Учебная программа «Робототехнический контроллер „Кибернетический“: моторы, сервоприводы, датчики»

Объём: 34 часа (1 урок по 45 минут в неделю, 34 учебные недели).

Структура: I полугодие — контроллер и датчики (уроки 1–16); II полугодие — моторы, сервоприводы, манипулятор (уроки 17–34).

Цель курса

Научить учащихся программировать робототехнический контроллер «Кибернетический», подключать к нему датчики, моторы и сервоприводы и собирать из них работающие устройства: от мигающего светодиода до робота, едущего по линии, и манипулятора.

Планируемые результаты

Учащиеся будут:

- знать устройство контроллера, назначение его узлов, правила питания и DIP-переключателя;
- писать программы в Arduino IDE для Arduino-части (ATmega328P) и ESP32;
- работать с цифровыми входами и выходами, шинами SPI, I2C, UART и шиной DXL (протокол Dynamixel 1.0);
- подключать и настраивать модули «Концевой выключатель», «Датчик линии», «Ультразвуковой дальномер»;
- управлять моторами с энкодерами (мощность, скорость, PID-регулятор) и сервоприводами Feetech ST3215;
- собирать, калибровать и программировать манипулятор;
- выполнять и защищать собственный проект.

Оборудование на рабочее место

- Робототехнический контроллер «Кибернетический», кабель USB-C, источник питания (аккумулятор с XT60 или блок питания с DC-разъёмом).
- Модули «Концевой выключатель», «Датчик линии», «Ультразвуковой дальномер», 3-пиновые кабели.
- Шасси с двумя моторами с энкодерами (II полугодие).
- Сервоприводы Feetech ST3215, манипулятор (на группу).
- Компьютер с Arduino IDE и браузером Chrome / Edge / Яндекс Браузер (для веб-настройщика).
- Поле с чёрной линией на белом фоне.

Памятка: DIP-переключатель SW12 (ON — слева)

№	ON = соединяет
1	ESP32 TX → Arduino RX
2	Arduino TX → ESP32 RX
3	ESP32 ↔ шина DXL
4	Arduino ↔ шина DXL
5	Включает светодиоды Arduino (выходы 74HC595)

Главные правила: **при прошивке Arduino DIP1 и DIP4 выключены; DIP3 и DIP4 одновременно не включать.**

Примеры кода

Примеры к урокам лежат в папке `lessons` этого репозитория:

- `arduino_ide/Ardu8_NN_Название/` — скетчи для Arduino IDE (папку целиком открыть в Arduino IDE);
- `ar_bbl/Ardu8_NN_Название.xmlar` — те же задачи в блочной среде AR BBL (там, где это возможно);
- `Примеры_по_урокам.xlsx` и `README.md` — список примеров и соответствие урокам обеих программ.

В каждом уроке строка **«Примеры»** показывает, какие примеры к нему относятся. Номер `NN` у одного и того же примера одинаковый в обеих программах. Примеры намеренно простые и подробно прокомментированы; в начале каждого — плата, разъём USB и положение DIP-переключателя.

I полугодие — контроллер и датчики (16 ч)

Модуль 0–1. Контроллер и встроенная периферия

Урок 1. Знакомство с контроллером «Кибернетический»

Цель: познакомиться с устройством контроллера и правилами безопасной работы.

Оборудование: контроллер (выключен), увеличенная карта разъёмов.

Ход урока:

1. Техника безопасности: питание, короткое замыкание, работа с аккумулятором — 10 мин.
2. Обзор платы по карте разъёмов — 20 мин.
3. Игра «Найди на плате»: преподаватель называет узел, учащиеся показывают — 10 мин.
4. Итоги — 5 мин.

Конспект:

- На плате два «мозга»: **Arduino-часть** (микроконтроллер ATmega328P, как в Arduino Uno) и **ESP32** (с Wi-Fi). У каждого свои 4 кнопки, 4 зелёных светодиода и RGB-светодиод: верхняя группа — Arduino, нижняя (у модуля ESP32) — ESP32.
- Два контроллера моторов STM8 управляют четырьмя моторами с энкодерами (разъёмы на правом краю). Также на плате есть датчик движения IMU, пищалка и интерфейс CAN.
- **Питание:** аккумулятор (XT60) или блок питания (DC) через красный выключатель. От USB питается только логика — моторы и сервоприводы при этом не работают.
- **USB-C на левом краю:** верхний — Arduino, нижний — ESP32. USB-C на верхнем краю — выход питания для внешних устройств.
- Сверху — 3-пиновые разъёмы для датчиков (GND, 5 В, сигнал), в центре — гребёнки как у Arduino Uno.
- Красный DIP-переключатель соединяет узлы платы между собой.
- Кнопка RESET перезапускает Arduino-часть, гасит её светодиоды и сбрасывает контроллеры моторов.

Задание: подписать узлы на распечатке платы.

Урок 2. Первая программа

Цель: научиться загружать программу в Arduino-часть контроллера.

Оборудование: контроллер, USB-C, компьютер с Arduino IDE.

Настройка: DIP1 = OFF, DIP4 = OFF; плата *Arduino Uno*; верхний USB-C.

Примеры: Arduino IDE — `Ardu8_01_Blink`, `Ardu8_02_SerialCounter`; AR BBL — `Ardu8_01_Blink`, `Ardu8_02_SerialCounter`.

Ход урока:

1. Знакомство с Arduino IDE: выбор платы и порта, кнопки «Проверить» и «Загрузить» — 10 мин.
2. Структура программы `setup()` / `loop()` — 10 мин.
3. Blink на D13 (светодиод HL1), изменение частоты мигания — 15 мин.
4. Монитор порта: вывод «Привет!» и счётчика — 10 мин.

Конспект:

- Скетч состоит из двух функций: `setup()` выполняется один раз при запуске, `loop()` повторяется бесконечно.
- `pinMode(13, OUTPUT)` настраивает вывод на выход, `digitalWrite(13, HIGH/LOW)` включает и выключает его, `delay(мс)` даёт паузу.
- На D13 стоит светодиод HL1. Он мигает и при обмене по SPI, потому что D13 — тактовая линия SPI.
- `Serial.begin(115200)` и `Serial.println()` выводят текст в монитор порта. Скорость в мониторе должна совпадать со скоростью в программе.
- Светодиоды HL3 и HL4 мигают при приёме и передаче по `Serial`.
- Почему DIP1 и DIP4 выключены: у Arduino один `Serial`, он общий для USB, шины моторов и ESP32. Если они подключены, прошивка не загрузится.

Задание: сделать сигнал SOS на HL1 (… — — — …).

Урок 3. Светодиоды через сдвиговый регистр 74HC595

Цель: понять, как сдвиговый регистр управляет восемью выходами по трём проводам.

Оборудование: контроллер, USB-C.

Настройка: DIP5 = ON.

Примеры: Arduino IDE — `Ardu8_04_Leds595_Running`, `Ardu8_05_Leds595_Binary`; AR BBL — `Ardu8_04_Leds595_Running`, `Ardu8_05_Leds595_Binary`.

Ход урока:

1. Двоичные числа и биты — 10 мин.
2. Как работает 74HC595: данные, такт, защёлка — 10 мин.
3. Зажечь один светодиод, затем бегущий огонь — 15 мин.
4. Двоичный счётчик 0–15 на LED1–LED4 — 10 мин.

Конспект:

- Сдвиговый регистр 74HC595 принимает байт по шине SPI (данные — D11, такт — D13) и выставляет его на 8 выходов, когда на защёлку (D8) подаётся импульс.
- Отправка байта: `digitalWrite(8, LOW); SPI.transfer(байт); digitalWrite(8, HIGH);`
- Биты: Q0–Q3 — LED1–LED4, Q4 — синий, Q5 — зелёный, Q6 — красный цвет RGB-светодиода, Q7 не используется.

- **Инверсная логика: 0 — горит, 1 — погашен.** Всё погасить: `0xFF`; зажечь только LED1: `0xFE` (`~0x01`).
- Без DIP5 = ON выходы регистра отключены, и светодиоды не горят при любой программе.
- Пины D8 и D13 заняты регистром, использовать их для датчиков нельзя.

Задание: бегущий огонь «туда и обратно».

Урок 4. Кнопки через 74HC165

Цель: научиться читать состояние кнопок через входной сдвиговый регистр.

Оборудование: контроллер, USB-C.

Настройка: DIP5 = ON.

Примеры: Arduino IDE — `Ardu8_07_Buttons165`, `Ardu8_08_ButtonCounter`; AR BBL — `Ardu8_07_Buttons165`, `Ardu8_08_ButtonCounter`.

Ход урока:

1. Как 74HC165 «фотографирует» кнопки и передаёт их по SPI — 10 мин.
2. Чтение байта кнопок, вывод в монитор порта в двоичном виде — 15 мин.
3. Кнопка → свой светодиод — 10 мин.
4. Дребезг контактов и антидребезг — 10 мин.

Конспект:

- Чтение кнопок: D10 → LOW (подключить регистр к линии MISO), импульс на D9 (LOW → HIGH, защёлкнуть состояние), `SPI.transfer(0)` — принять байт, D10 → HIGH.
- Кнопки BUT1–BUT4 (слева направо) — биты 0–3. **Нажатие даёт 1.**
- Проверка кнопки: `if (buttons & (1 << 0))` — нажата BUT1.
- При нажатии контакты «дребезжат» несколько миллисекунд. Простое решение — прочитать ещё раз через 20–30 мс.
- Перенос кнопок на светодиоды с учётом инверсии: `leds = ~buttons`.

Задание: счётчик нажатий BUT1, результат в двоичном виде на LED1–LED4, сброс по BUT4.

Урок 5. RGB-светодиод и конечный автомат

Цель: освоить смешение цветов и программирование по состояниям.

Оборудование: контроллер, USB-C.

Настройка: DIP5 = ON.

Примеры: Arduino IDE — `Ardu8_10_RGB_Rainbow`, `Ardu8_11_TrafficLight`; AR BBL — `Ardu8_10_RGB_Rainbow`, `Ardu8_11_TrafficLight`.

Ход урока:

1. Смешение цветов: красный + зелёный = жёлтый и т. д. — 10 мин.
2. Понятие «конечный автомат»: состояния и переходы — 10 мин.
3. Светофор с кнопкой пешехода — 20 мин.
4. Итоги — 5 мин.

Конспект:

- RGB-светодиод — три светодиода в одном корпусе: Q4 синий, Q5 зелёный, Q6 красный. Включая их в разных сочетаниях, получаем 7 цветов.
- Конечный автомат: программа в каждый момент находится в одном состоянии (ЗЕЛЁНЫЙ, ЖЁЛТЫЙ, КРАСНЫЙ) и переходит в другое по событию (прошло время, нажата кнопка).
- Состояние хранится в переменной, переходы описываются через `switch`.
- Чтобы кнопка работала во время ожидания, вместо `delay()` используется `millis()`: «если прошло больше N мс — перейти дальше».

Задание: добавить мигание зелёного перед сменой на жёлтый.

Урок 6. Датчик движения IMU ICM-20948 и шина I2C

Цель: познакомиться с шиной I2C и встроенным акселерометром.

Оборудование: контроллер, USB-C.

Примеры: Arduino IDE — `Ardu8_13_I2C_Scanner`, `Ardu8_14_IMU_Accel`.

Ход урока:

1. Шина I2C: два провода (SDA — A4, SCL — A5), адреса устройств — 10 мин.
2. Сканер I2C: найти устройства на шине — 10 мин.
3. Чтение акселерометра, наклон в плоттере — 20 мин.
4. Итоги — 5 мин.

Конспект:

- По шине I2C к одному контроллеру можно подключить много устройств. Каждое имеет свой адрес.
- IMU ICM-20948 имеет адрес **0x68**. Внутри него — акселерометр, гироскоп и магнитометр.
- Акселерометр измеряет ускорение по трём осям. Когда плата лежит неподвижно, он «чувствует» только силу тяжести (1 g), поэтому по нему можно определить наклон.
- Плоттер Arduino IDE рисует графики, если выводить числа через запятую или пробел в одной строке.
- Те же линии I2C выведены на разъём XP5 и RJ12 XS9.

Задание: зажигать LED1 при наклоне влево и LED4 при наклоне вправо.

Урок 7. Практикум по модулю 1

Цель: закрепить работу со светодиодами, кнопками и IMU.

Оборудование: контроллер, USB-C.

Примеры: Arduino IDE — `Ardu8_12_ReactionGame`, `Ardu8_17_IMU_BubbleLevel`; AR BBL — `Ardu8_12_ReactionGame`.

Ход урока: выбор задания — 5 мин, выполнение — 30 мин, демонстрация — 10 мин.

Конспект:

- «Игра на реакцию»: через случайное время (`random()`) загорается RGB, побеждает тот, кто первым нажмёт свою кнопку (BUT1 или BUT4). Время реакции выводится в монитор порта.
- «Уровень-пузырёк»: «пузырёк» (горящий светодиод) смещается в сторону наклона, в горизонтальном положении RGB горит зелёным.
- Порядок работы над задачей: описать состояния → нарисовать схему переходов → написать код → проверить.

Задание: доработать игру до трёх раундов со счётом.

Модуль 2. Датчики Kirus

Урок 8. Модуль «Концевой выключатель»

Цель: научиться читать цифровой датчик.

Оборудование: контроллер, модуль «Концевой выключатель», 3-пиновый кабель.

Подключение: модуль → D7 (XP21). Скетч `Ardu8_Kirus_uSW`.

Примеры: Arduino IDE — `Ardu8_18_Endstop`, `Ardu8_19_Bumper`; AR BBL — `Ardu8_18_Endstop`, `Ardu8_19_Bumper`.

Ход урока:

1. Устройство модуля, подтягивающий резистор — 10 мин.
2. Подключение, проверка по светодиоду модуля — 5 мин.
3. Разбор и загрузка `Ardu8_Kirus_uSW` — 15 мин.
4. Своя доработка — 15 мин.

Конспект:

- Концевой выключатель — кнопка, которая срабатывает, когда механизм доходит до края или робот касается препятствия.
- В модуле стоит подтягивающий резистор: **не нажат — HIGH, нажат — LOW.** Зелёный светодиод модуля горит при нажатии.
- Разъёмы модуля X2 и X4 одинаковые и совпадают с 3-пиновыми разъёмами контроллера (GND, 5 В, сигнал).
- Считаем «событие нажатия», а не состояние: запоминаем прошлое значение и ловим переход HIGH → LOW.
- В скетче: LED1 и RGB показывают нажатие, ведётся счётчик нажатий с антидребезгом, BUT1 сбрасывает счёт.

Задание: после 10 нажатий пищать или мигать красным.

Урок 9. Модуль «Датчик линии»

Цель: понять принцип работы оптического датчика и настроить его на поле.

Оборудование: контроллер, модуль «Датчик линии», 2 кабеля, поле с линией.

Подключение: S1 (X4) → D3 (XP17), S2 (X2) → D4 (XP18). Скетч `Ardu8_Kirus_Line`.

Примеры: Arduino IDE — `Ardu8_20_LineSensor`, `Ardu8_21_LinePositions`; AR BBL — `Ardu8_20_LineSensor`, `Ardu8_21_LinePositions`.

Ход урока:

1. Отражение ИК-света от белой и чёрной поверхности — 10 мин.
2. Подключение, настройка порога подстроечником R7 — 15 мин.
3. Скетч `Ardu8_Kirus_Line`, вывод S1/S2 — 15 мин.
4. Итоги — 5 мин.

Конспект:

- В модуле два оптрона TCRT5000: ИК-светодиод светит вниз, фототранзистор ловит отражённый свет. Белое отражает хорошо, чёрное — плохо.
- Компаратор превращает сигнал в цифровой: **белое — LOW (горит зелёный светодиод канала), чёрное — HIGH.**
- Порог срабатывания общий для двух каналов и задаётся подстроечником R7. Настраивать на том поле и при том освещении, где будет ездить робот.
- Высота датчика над полем сильно влияет на работу: обычно несколько миллиметров.
- Скетч показывает каналы на LED1/LED2 и цветом RGB.

Задание: определить по двум каналам и вывести в монитор: «линия слева», «линия справа», «на линии», «линии нет».

Урок 10. Модуль «Ультразвуковой дальномер»

Цель: научиться измерять расстояние ультразвуком.

Оборудование: контроллер, модуль «Ультразвуковой дальномер», 2 кабеля, линейка.

Подключение: TRIG (X2) → D5 (XP19), ECHO (X4) → D6 (XP20). Скетч `Ardu8_Kirus_Ultrasonic`.

Примеры: Arduino IDE — `Ardu8_22_Ultrasonic`, `Ardu8_23_UltrasonicFilter`; AR BBL — `Ardu8_22_Ultrasonic`.

Ход урока:

1. Принцип эхолокации — 10 мин.
2. Подключение и загрузка скетча — 10 мин.
3. Опыт: сравнить показания с линейкой на 10, 30, 50 см — 15 мин.
4. Медианный фильтр, плоттер — 10 мин.

Конспект:

- Импульс 10 мкс на TRIG запускает «писк» на 40 кГц. На выходе ECHO появляется импульс, длительность которого равна времени полёта звука туда и обратно.
- `pulseIn(6, HIGH, 30000)` измеряет длительность импульса в микросекундах (с ограничением ожидания).
- Звук проходит 1 см примерно за 29 мкс, туда и обратно — за 58 мкс. Отсюда **расстояние, см = мкс / 58**.
- Случайные выбросы убирает медианный фильтр: берём три измерения и выбираем среднее по величине.
- Светодиоды модуля: ECHO мигает во время измерений, TRIG горит, когда программа держит линию в LOW.
- В скетче расстояние показывается шкалой на LED1–LED4, зонами цвета RGB и графиком в плоттере.

Задание: найти минимальное и максимальное расстояние, которое уверенно измеряет датчик.

Урок 11. Все датчики вместе

Цель: подключить все модули одновременно и вывести их показания.

Оборудование: контроллер, три модуля, 5 кабелей.

Подключение: как на уроках 8–10. Скетч `Ardu8_Kirus_All`.

Примеры: Arduino IDE — `Ardu8_24_AllSensors`; AR BBL — `Ardu8_24_AllSensors`.

Ход урока:

1. Схема подключения всех модулей — 10 мин.
2. Загрузка `Ardu8_Kirus_All`, плоттер — 15 мин.
3. Опыты с каждым датчиком, наблюдение графиков — 15 мин.
4. Итоги — 5 мин.

Конспект:

- Модули занимают D3–D7 и не мешают друг другу. D2 оставлен свободным: он понадобится библиотеке моторов.
- Плоттер понимает подписи вида `Distance_cm:25 Line_S1:0 Line_S2:1 Endstop:1` — каждая подпись становится отдельной линией графика.
- Если программа долго ждёт ECHO, остальные датчики опрашиваются реже. Поэтому важно ограничивать время ожидания `pulseIn`.

Задание: нарисовать в тетради график, который получился при поднесении руки к дальномеру.

Урок 12. Проект «Парктроник»

Цель: применить датчик расстояния в законченном устройстве.

Оборудование: контроллер, модуль «Ультразвуковой дальномер».

Примеры: Arduino IDE — `Ardu8_25_Parktronic`; AR BBL — `Ardu8_25_Parktronic`.

Ход урока: постановка задачи — 5 мин, разработка — 30 мин, демонстрация — 10 мин.

Конспект:

- Парктроник предупреждает водителя о препятствии: чем ближе препятствие, тем чаще сигнал.
- Зоны: больше 50 см — зелёный, 20–50 см — жёлтый, меньше 20 см — красный.
- Пищалка стоит на ESP32. В Arduino-части звук можно заменить быстрым миганием, а со звуком проект доработать после урока 13.
- Частоту мигания удобно вычислять по расстоянию: `период = расстояние * 10` мс.

Задание: добавить «режим тишины» по кнопке.

Модуль 3. ESP32

Урок 13. ESP32 — вторая «голова» платы

Цель: научиться программировать ESP32 на контроллере.

Оборудование: контроллер, USB-C, Arduino IDE с пакетом плат ESP32.

Настройка: нижний USB-C, плата *ESP32 Wrover Module*.

Примеры: Arduino IDE — `Ardu8_26_ESP32_Leds`, `Ardu8_27_ESP32_Piano`, `Ardu8_28_ESP32_Melody`; AR BBL — `Ardu8_26_ESP32_Leds`, `Ardu8_27_ESP32_Piano`.

Ход урока:

1. Отличия ESP32 от Arduino-части: 3,3 В, Wi-Fi, много выводов — 10 мин.
2. Загрузка программы, кнопки BOOT и EN — 10 мин.
3. Светодиоды и кнопки ESP32 — 15 мин.
4. Мелодия на пищалке — 10 мин.

Конспект:

- ESP32 подключается к нижнему USB-C. Если загрузка не начинается, нажать и держать BOOT в момент подключения к порту; EN — перезапуск.
- Светодиоды ESP32 подключены напрямую к выводам: IO27, IO14, IO12, IO13 — зелёные; IO15, IO2, IO18 — красный, зелёный, синий. **LOW — горит.**
- Кнопки: IO34, IO35, IO25, IO26 (BUT1–BUT4). **Нажатие — HIGH.** Слева направо они стоят в обратном порядке: BUT4, BUT3, BUT2, BUT1.
- Пищалка на IO19: `tone(19, 440, 200)` — нота ля 200 мс.
- ESP32 работает от 3,3 В: датчики с выходом 5 В подключать к нему напрямую нельзя.

Задание: «пианино» — каждая кнопка играет свою ноту.

Урок 14. Связь Arduino ↔ ESP32 ⚠️

Цель: передать показания датчиков из Arduino-части в ESP32 и показать их на веб-странице.

Оборудование: контроллер, все модули Kirus, смартфон.

Настройка: DIP2 = ON. Скетчи `Ardu8_Kirus_Sender` (Arduino) и `Ardu8_esp_Kirus_Receiver` (ESP32).

Примеры: Arduino IDE — `Ardu8_30_UART_Sender`, `Ardu8_31_UART_Receiver`, `Ardu8_32_UART_WebStation`; AR BBL — `Ardu8_30_UART_Sender`.

Ход урока:

1. Последовательный порт UART: передача по одному проводу — 10 мин.
2. Формат сообщения, загрузка двух скетчей — 15 мин.
3. Подключение к Wi-Fi-точке, страница 192.168.4.1 — 15 мин.
4. Итоги — 5 мин.

Конспект:

- Датчики подключены к Arduino-части, а Wi-Fi есть только у ESP32. Поэтому Arduino отправляет данные в ESP32 по UART (DIP2 соединяет Arduino TX с ESP32 RX).
- Arduino раз в несколько десятков миллисекунд отправляет строку `K,S1,S2,см,uSW` на скорости 115200.
- ESP32 принимает её через `Serial2.begin(115200, SERIAL_8N1, 22, 23)` и разбирает по запятым.
- ESP32 создаёт Wi-Fi-точку «Ardu8-Kirus». Подключившись к ней, на телефоне можно открыть http://192.168.4.1 и увидеть показания.
- Прошивать Arduino лучше при DIP2 = OFF.

Задание: сделать звуковой сигнал ESP32 при срабатывании концевого выключателя.

⚠️ Примеры для этого урока на железе ещё не проверены — прогнать до урока.

Урок 15. Зачёт за полугодие

Цель: проверить умения по модулям 1–3.

Ход урока: получение задания — 5 мин, выполнение — 35 мин, показ — 5 мин.

Конспект (критерии):

- Правильные подключение и положение DIP-переключателя.
- Программа загружается и работает по заданию.

- Учащийся может объяснить свой код.
- Примеры заданий: «Кодовый замок на кнопках», «Сигнализация на концевом выключателе», «Индикатор расстояния».

Урок 16. Резерв

Отработка пропущенных тем, доработка проектов, пересдача зачёта.

II полугодие — моторы, сервоприводы, манипулятор (18 ч)

Модуль 4. Моторы

Урок 17. Шина DXL и протокол Dynamixel 1.0

Цель: понять, как контроллер управляет модулями моторов по шине.

Оборудование: контроллер с питанием, 2 мотора с энкодерами.

Настройка: DIP3 = ON, DIP4 = OFF. Скетч `Ardu8_esp_motors_test` (ESP32).

Примеры: Arduino IDE — `Ardu8_33_DXL_Ping`.

Ход урока:

1. Зачем нужна шина: много устройств на одном проводе — 10 мин.
2. Устройство пакета Dynamixel, расчёт контрольной суммы — 15 мин.
3. Поиск модулей скетчем, проверка по светодиодам драйверов — 15 мин.
4. Итоги — 5 мин.

Конспект:

- Моторами управляют два контроллера STM8. Они подключены к шине DXL — одному проводу, по которому данные идут в обе стороны по очереди.
- Каждое устройство на шине имеет свой номер (ID). Модули моторов: **ID 71 и 72.**
- Разъёмы моторов на правом краю, сверху вниз: 71/M1, 71/M2, 72/M1, 72/M2.
- Пакет: `FF FF ID LEN INST PARAMS… CHK`. Команды: PING (1) — «ты здесь?», READ (2) — прочитать, WRITE (3) — записать. Контрольная сумма: `CHK = ~(ID + LEN + INST + PARAMS)`, младший байт.
- Скорость шины для модулей моторов — 57600. ESP32: `Serial2.begin(57600, SERIAL_8N1, 33, 32)`.
- Моторы работают только при питании от XT60/DC и включённом выключателе.

Задание: посчитать вручную контрольную сумму пакета PING для ID 71.

Урок 18. Мощность и энкодеры

Цель: управлять мотором по мощности и измерять пройденный путь энкодером.

Оборудование: контроллер, шасси с моторами.

Настройка: DIP3 = ON, DIP4 = OFF.

Примеры: Arduino IDE — `Ardu8_34_DXL_Power`, `Ardu8_35_DXL_Encoder`.

Ход урока:

1. Регистры модуля, ШИМ — 10 мин.

2. Запуск мотора с разной мощностью и в разные стороны — 10 мин.
3. Энкодер: сколько тиков на оборот колеса — 15 мин.
4. Проехать ровно N оборотов — 10 мин.

Конспект:

- У модуля есть «ячейки памяти» — регистры. Записываем число в регистр — модуль выполняет команду.
- **Мощность:** регистр 28 (M1) и 30 (M2), от −1023 до 1023. Знак задаёт направление, 0 — стоп. Запись мощности включает режим ШИМ.
- ШИМ — быстрое включение и выключение питания. Чем больше доля времени «включено», тем быстрее вращается мотор.
- **Энкодер** считает импульсы при вращении вала. Регистры 32 (M1) и 36 (M2), число со знаком.
- Количество тиков на оборот колеса определяем опытом: отмечаем колесо, поворачиваем на 10 оборотов, делим.
- Путь колеса = обороты × π × диаметр.

Задание: проехать 1 м и измерить ошибку рулеткой.

Урок 19. Режим скорости и PID-регулятор

Цель: понять, зачем нужен регулятор, и настроить его.

Оборудование: контроллер, шасси.

Примеры: Arduino IDE — `Ardu8_37_DXL_SpeedPID`.

Ход урока:

1. Опыт: при одинаковой мощности под нагрузкой колесо замедляется — 10 мин.
2. Идея регулятора с обратной связью — 10 мин.
3. Режим скорости, опыты с коэффициентами P и I — 20 мин.
4. Итоги — 5 мин.

Конспект:

- В режиме ШИМ скорость зависит от нагрузки и заряда аккумулятора. Регулятор сам меняет мощность, чтобы скорость оставалась заданной.
- **Заданная скорость:** регистр 40 (M1) и 42 (M2), в тиках за 100 мс. Запись включает режим скорости. Текущая скорость — регистры 34 и 38.
- Ошибка = заданная скорость − текущая. P (регистр 47, по умолчанию 16) реагирует на ошибку сейчас, I (48, по умолчанию 32) — на накопленную ошибку, D (49, по умолчанию 0) — на её изменение.
- Слишком большой P даёт рывки и колебания, слишком маленький — вялый разгон. Без I скорость немного не дотягивает до заданной.
- Регистр 24 ограничивает максимальную мощность в режиме скорости. Регистры PID сбрасываются при выключении питания.

Задание: построить в плоттере графики заданной и текущей скорости при двух разных P.

Урок 20. Моторы с Arduino через DxlMaster

Цель: управлять моторами из Arduino-части готовой библиотекой.

Оборудование: контроллер, шасси.

Настройка: прошивать при DIP4 = OFF, затем DIP4 = ON, DIP1 = OFF, DIP3 = OFF. Скетч `Ardu8_dxlmaster_test`.

Примеры: Arduino IDE — `Ardu8_38_DxlMaster_Buttons`; AR BBL — `Ardu8_38_DxlMaster_Buttons`.

Ход урока:

1. Библиотека DxlMaster — 10 мин.
2. Загрузка тестового скетча, порядок переключения DIP — 10 мин.
3. Управление шасси кнопками: вперёд, назад, повороты — 20 мин.
4. Итоги — 5 мин.

Конспект:

- Создаём модуль: `DynamixelDevice drv(71);`. Запись мощности: `drv.write(28, (int16_t)400);`. Чтение энкодера: `drv.read(32, enc);`.
- Шина висит на том же `Serial`, что и USB, поэтому прошивать нужно с выключенным DIP4. `Serial.print` при работе попадает на шину — для отладки используем светодиоды.
- Библиотека занимает вывод **D2**, поэтому датчики к нему не подключаем.
- Повороты: «танковый» (колёса в разные стороны) — на месте, плавный — одно колесо медленнее другого.
- Если робот едет не туда, меняем знак мощности одного из моторов в программе.

Задание: проехать «квадрат» по времени.

Урок 21. Проект «Робот по линии»: сборка

Цель: подготовить шасси к езде по линии.

Оборудование: шасси, модуль «Датчик линии», модули дальномера и концевого выключателя, крепёж.

Примеры: Arduino IDE — `Ardu8_24_AllSensors`, `Ardu8_38_DxlMaster_Buttons`; AR BBL — `Ardu8_24_AllSensors`, `Ardu8_38_DxlMaster_Buttons`.

Ход урока: установка датчиков — 20 мин, проверка направлений моторов — 15 мин, настройка R7 на поле — 10 мин.

Конспект:

- Датчик линии ставится впереди по центру, низко над полем. Дальномер смотрит вперёд, концевой выключатель — бампер.
- Подключение: S1 → D3, S2 → D4, TRIG → D5, ECHO → D6, концевой выключатель → D7. D2 не занимать.
- Проверка: команда «вперёд» должна вращать оба колеса вперёд. Нужно определить, какой канал (S1 или S2) оказался слева.

Задание: записать в тетрадь знаки моторов и сторону S1 для своего робота.

Урок 22. Робот по линии: алгоритм ⚠️

Цель: запрограммировать движение по линии двумя датчиками.

Оборудование: собранный робот, поле.

Скетч: `Ardu8_Kirus_LineFollower` (DIP4 = ON).

Примеры: Arduino IDE — `Ardu8_42_LineFollower`; AR BBL — `Ardu8_42_LineFollower`.

Ход урока:

1. Алгоритм «два датчика» — 10 мин.
2. Разбор скетча, подстановка знаков моторов и стороны S1 — 10 мин.
3. Заезды и настройка скорости — 25 мин.

Конспект:

- Датчики стоят по краям линии. Оба на белом — едем прямо. Левый на чёрном — поворачиваем влево. Правый на чёрном — вправо. Оба на чёрном — перекрёсток или стоп-линия.
- Чем выше скорость, тем чаще робот вылетает с линии на поворотах. Поворот можно делать плавнее, уменьшая скорость внутреннего колеса.
- Если робот «виляет» — снизить скорость или разницу скоростей колёс при повороте.

Задание: пройти трассу без вылетов, записать лучшее время.

⚠️ Примеры — черновики, знаки моторов и сторону S1 подобрать на реальном шасси.

Урок 23. Робот по линии: препятствия

Цель: добавить реакцию на препятствие.

Оборудование: робот, поле, препятствие (кубик).

Примеры: Arduino IDE — `Ardu8_43_LineObstacle`; AR BBL — `Ardu8_43_LineObstacle`.

Ход урока: обсуждение — 10 мин, программирование — 20 мин, тренировочный заезд — 15 мин.

Конспект:

- Дальномер: если расстояние меньше 15 см — остановиться и ждать, пока препятствие уберут.
- Концевой выключатель как бампер: при касании — стоп и отъезд назад.
- Проверка датчиков не должна надолго останавливать главный цикл: ожидание ECHO ограничено по времени.

Задание: робот объезжает препятствие и возвращается на линию (по желанию).

Модуль 5. Сервоприводы Feetech ST3215

Урок 24. Сервопривод ST3215 и веб-настройщик

Цель: подключить сервопривод, найти его на шине и назначить ID.

Оборудование: контроллер с питанием, сервопривод ST3215, компьютер с Chrome / Edge / Яндекс Браузером.

Настройка: DIP3 = ON, DIP4 = OFF, привод в XP2 или XP3. Скетчи `Ardu8_ST3215_Scan`, `Ardu8_ST3215_SetID`, `Ardu8_Feetech_Bridge`; страница `feetech_tool.html`.

Примеры: примеры библиотеки FeetechSTS — `Ardu8_ST3215_Scan`, `Ardu8_ST3215_SetID`; мост `Ardu8_Feetech_Bridge` и страница `feetech_tool.html`.

Ход урока:

1. Чем сервопривод отличается от мотора — 10 мин.
2. Поиск привода скетчем, смена ID — 15 мин.

3. Мост и веб-настройщик, раздел «Один привод» — 15 мин.
4. Итоги — 5 мин.

Конспект:

- Сервопривод сам удерживает заданный угол: внутри мотор, редуктор, датчик угла и контроллер.
- ST3215 подключается к той же шине DXL, что и модули моторов, но работает на скорости **1 000 000 бит/с** и использует свою таблицу регистров.
- Питание привода идёт от входа без стабилизации: **напряжение аккумулятора должно подходить приводу** (бывают на 7,4 и на 12 В).
- Заводской ID — 1. Чтобы сменить ID, на шине должен быть только один привод. **ID 71 и 72 приводам не назначать** — они заняты модулями моторов.
- Веб-настройщик работает через мост `Ardu8_Feetech_Bridge` в ESP32: режим «Ardu8», порт нижнего USB-C. Монитор порта Arduino IDE перед этим нужно закрыть.

Задание: назначить приводу ID, выданный преподавателем, и подписать его.

Урок 25. Режим позиции и обратная связь

Цель: поворачивать вал на заданный угол и читать состояние привода.

Оборудование: контроллер, сервопривод.

Скетч: `Ardu8_ST3215_Position`.

Примеры: Arduino IDE — `Ardu8_46_ST3215_Metronome`, `Ardu8_47_ST3215_Feedback`; примеры библиотеки FeetechSTS — `Ardu8_ST3215_Position`.

Ход урока:

1. Шкала позиции: 4096 шагов на оборот — 10 мин.
2. Загрузка скетча, работа с кнопками — 15 мин.
3. Чтение нагрузки, температуры, тока — 15 мин.
4. Итоги — 5 мин.

Конспект:

- Один оборот — 4096 шагов, 2048 — середина (180°). 1° ≈ 11,4 шага.
- Подключение к шине: `bus.begin(Serial2, 1000000, 33, 32)`. Привод: `FeetechServo servo(bus, 1);`.
- При старте вызываем **`servo.hold()`**: он записывает текущий угол как цель. Иначе вал может резко дёрнуться к старой цели.
- `servo.moveToDeg(270, 2000, 50)` — угол, скорость (шагов/с), ускорение.
- Кнопки ESP32 в примере: BUT1 — 90°, BUT2 — 180°, BUT3 — 270°, BUT4 — отпустить вал. RGB зелёный во время движения.
- Обратная связь: `readLoad()`, `readTemperature()`, `readCurrent()`, `readVoltage()`. По росту нагрузки можно понять, что вал упёрся.

Задание: «метроном» — вал качается между 150° и 210°.

Урок 26. Режим колеса и несколько приводов ⚠️

Цель: использовать привод как мотор и управлять несколькими приводами одновременно.

Оборудование: контроллер, 2–3 сервопривода с разными ID.

Скетчи: `Ardu8_ST3215_Wheel`, `Ardu8_ST3215_Sync`.

Примеры: Arduino IDE — `Ardu8_49_ST3215_Conveyor`, `Ardu8_50_ST3215_Mirror`; примеры библиотеки FeetechSTS — `Ardu8_ST3215_Wheel`, `Ardu8_ST3215_Sync`.

Ход урока: режим колеса — 15 мин, синхронное движение — 20 мин, итоги — 10 мин.

Конспект:

- Режим колеса: `servo.setMode(STS_MODE_WHEEL)` и `servo.setSpeed(шагов/с)` — привод вращается непрерывно, как мотор.
- Чтобы несколько приводов пришли в цель одновременно, используют одну общую команду SYNC WRITE: `bus.syncMoveTo(...)`.
- Если отправлять команды по очереди, приводы начинают движение с задержкой.

Задание: два привода качаются «в зеркало».

⚠️ Скетчи с SYNC-командами на железе ещё не проверены (в веб-настройщике SYNC WRITE проверен).

Модуль 6. Манипулятор

Урок 27. Манипулятор: подготовка приводов

Цель: подготовить 7 приводов к сборке.

Оборудование: 7 сервоприводов ST3215, фланцы, маркер, контроллер, веб-настройщик.

Ход урока: устройство манипулятора — 10 мин, работа с помощником «Подготовка приводов» — 30 мин, итоги — 5 мин.

Конспект:

- Суставы снизу вверх: 1 — поворот основания, 2 — плечо, 3 — локоть, 4 — поворот предплечья, 5 — наклон кисти, 6 — поворот кисти, 7 — схват. ID привода равен номеру сустава.
- Новые приводы имеют ID 1, поэтому подключаем их строго по одному.
- Помощник: найти привод → записать ID и скорость 1 Мбит/с → повернуть вал в 2048 (середина) → надеть фланец.
- Фланец: 4 отверстия крестом «+» вдоль и поперёк корпуса, центральный винт M3, риска маркером на фланце и корпусе, подпись «ID N».

Задание: подготовить все приводы своей группы.

Урок 28. Манипулятор: сборка

Цель: собрать манипулятор, не сбив нулевые положения приводов.

Оборудование: подготовленные приводы, детали манипулятора, крепёж, инструмент.

Ход урока: разбор сборочной схемы — 10 мин, сборка — 30 мин, проверка — 5 мин.

Конспект:

- Собираем в «позе модели»: плечо вертикально, предплечье горизонтально вперёд, кисть вниз. Риски на фланцах и корпусах должны совпадать.
- Шлицы фланца дают шаг 14,4°: если промахнуться на один зубец, сустав будет смещён.
- Холостые фланцы (приводы 2–5) вращаются свободно, их ставят при сборке звена.
- Шестерни схвата ставят при полностью раскрытых губках.

- Кабели укладывают так, чтобы они не натягивались при движении суставов.

Задание: проверить вручную, что каждый сустав свободно двигается во всём диапазоне.

Урок 29. Калибровка и позы

Цель: откалибровать суставы и запрограммировать последовательность поз.

Оборудование: манипулятор, контроллер, веб-настройщик (раздел «Манипулятор»).

Ход урока: калибровка — 15 мин, ограничения суставов — 10 мин, запись и проигрывание поз — 20 мин.

Конспект:

- «Ноль = сейчас» записывает текущее положение в привод как середину шкалы, после чего 0° сустава соответствует 2048.
- Инверсия меняет направление вращения сустава, если он поворачивается «не в ту сторону».
- Пределы защищают манипулятор от ударов о себя и о стол.
- Поза — набор углов всех суставов. Позы запоминаются, переставляются и проигрываются по кругу. Все суставы приходят в позу одновременно за заданное «время перехода».
- Настройки хранятся в браузере. Их нужно сохранить в JSON-файл, чтобы не потерять.

Задание: записать 4 позы: «домой», «над кубиком», «взять», «положить».

Итоговый проект

Урок 30. Итоговый проект: разработка

Цель: спланировать и начать выполнение проекта.

Конспект:

- Базовая задача: «Перенеси кубик из зоны А в зону В» — манипулятор. Другие варианты: робот по линии с остановками, «Умная станция».
- План проекта: цель → оборудование → алгоритм (схема) → распределение ролей → критерии успеха.
- Последовательность для манипулятора: «домой» → над кубиком → опуститься → сжать схват → подняться → перенести → опустить → разжать → «домой».

Урок 31. Итоговый проект: отладка

Конспект:

- Отлаживаем по частям: сначала каждая поза отдельно, затем переходы, затем весь цикл.
- Типичные проблемы: кубик выскальзывает (схват сжимается слишком сильно или слабо), сустав перегревается (большая нагрузка, малое время перехода), манипулятор задевает стол (пределы).
- Записываем, что исправили, — это пригодится на защите.

Урок 32. Резерв

Доработка проектов, подготовка к защите.

Урок 33. Соревнование / защита проектов

Конспект (регламент):

- Заезды по линии: время прохождения трассы, штраф за вылет.
- Манипулятор: число кубиков, перенесённых за 2 минуты.
- Защита (3–5 минут): цель, устройство, алгоритм, трудности и как их решили, демонстрация.

Урок 34. Подведение итогов

Обсуждение результатов года, награждение, планы на продолжение (расширенная программа на 68 часов).

Примечания

⚠️ — уроки опираются на скетчи, которые ещё не проверены на железе. Их нужно прогнать на железе до начала второго полугодия, а урок 14 — до декабря. Примеры к урокам с IMU (`Ardu8_13`–`Ardu8_17`) и все новые примеры Arduino IDE из папки `lessons` компилируются, но на железе ещё не проверены. Примеры AR BBL, совпадающие с набором `ar_bbl_examples`, проверены на контроллере 30.10.2026.